C. Books Queries

time limit per test 2 seconds
memory limit per test 256 megabytes
input standard input
output standard output

Rating: 1400 | Tags: implementation | https://codeforces.com/problemset/problem/1066/C

You have got a shelf and want to put some books on it.

You are given q queries of three types:

1. L *id* — put a book having index *id* on the shelf to the left from the leftmost existing book;
2. R *id* — put a book having index *id* on the shelf to the right from the rightmost existing book;
3. ? *id* — calculate the minimum number of books you need to pop from the left or from the right in such a way that the book with index *id* will be leftmost or rightmost.

You can assume that the first book you will put can have any position (it does not matter) and queries of type 3 are always valid (it is guaranteed that the book in each such query is already placed). You can also assume that you don't put the same book on the shelf twice, so *id*s don't repeat in queries of first two types.

Your problem is to answer all the queries of type 3 in order they appear in the input.

Note that after answering the query of type 3 all the books remain on the shelf and the relative order of books does not change.

If you are Python programmer, consider using PyPy instead of Python when you submit your code.

Input

The first line of the input contains one integer q ($1 \le q \le 2 \cdot 10^5$) — the number of queries.

Then q lines follow. The i-th line contains the i-th query in format as in the problem statement. It is guaranteed that queries are always valid (for query type 3, it is guaranteed that the book in each such query is already placed, and for other types, it is guaranteed that the book was not placed before).

It is guaranteed that there is at least one query of type 3 in the input.

In each query the constraint $1 \le id \le 2 \cdot 10^5$ is met.

Output

Print answers to queries of the type 3 in order they appear in the input.

Examples

Input

```
8
L 1
R 2
R 3
? 2
L 4
? 1
L 5
? 1
```

Output

```
1
1
2
```

Input

```
10
L 100
R 100000
R 123
L 101
? 123
L 10
R 115
? 100
R 110
? 115
```

Output

```
0
2
1
```

Note

Let's take a look at the first example and let's consider queries:

1. The shelf will look like [1];
2. The shelf will look like [1, 2];
3. The shelf will look like [1, 2, 3];
4. The shelf looks like [1, **2**, 3] so the answer is 1;
5. The shelf will look like [4, 1, 2, 3];
6. The shelf looks like [4, **1**, 2, 3] so the answer is 1;
7. The shelf will look like [5, 4, 1, 2, 3];
8. The shelf looks like [5, 4, **1**, 2, 3] so the answer is 2.

Let's take a look at the second example and let's consider queries:

1. The shelf will look like [100];
2. The shelf will look like [100, 100000];
3. The shelf will look like [100, 100000, 123];
4. The shelf will look like [101, 100, 100000, 123];
5. The shelf looks like [101, 100, 100000, **123**] so the answer is 0;

6. The shelf will look like [10, 101, 100, 100000, 123];
7. The shelf will look like [10, 101, 100, 100000, 123, 115];
8. The shelf looks like [10, 101, **100**, 100000, 123, 115] so the answer is 2;
9. The shelf will look like [10, 101, 100, 100000, 123, 115, 110];
10. The shelf looks like [10, 101, 100, 100000, 123, **115**, 110] so the answer is 1.